

3.1 Implementasi Backend

Pada tahap ini dilakukan proses pembangunan backend aplikasi **WisataCui** menggunakan framework **NestJS** dengan bahasa pemrograman **TypeScript**. Backend bertugas sebagai penyedia layanan (service) yang akan diakses oleh frontend melalui REST API.

Selain itu digunakan **Prisma ORM** sebagai penghubung antara aplikasi dengan database **MySQL**, sehingga proses pengolahan data menjadi lebih mudah, aman, dan terstruktur.

3.2 Persiapan Lingkungan Pengembangan

Sebelum melakukan implementasi backend, dilakukan instalasi beberapa dependency yang dibutuhkan.

Software yang digunakan

No Software	Fungsi
1 Node.js	Menjalankan JavaScript Runtime
2 NestJS CLI	Framework Backend
3 Prisma ORM	ORM Database
4 MySQL	Database
5 Postman	Pengujian REST API
6 Visual Studio Code Code Editor	

3.3 Konfigurasi Database

Database yang digunakan adalah **MySQL** dengan nama database:

wisatacui

Konfigurasi koneksi database disimpan pada file **.env**.

Contoh konfigurasi:

```
DATABASE_URL="mysql://root:password@localhost:3306/wisatacui"
```

Setelah konfigurasi selesai dilakukan sinkronisasi database menggunakan Prisma.

```
npx prisma db push
```

Perintah tersebut digunakan untuk membuat seluruh tabel pada database sesuai dengan schema Prisma.

3.4 Desain Database

Backend menggunakan empat tabel utama yaitu:

- User
- Wisata
- Category
- Review

Relasi antar tabel adalah sebagai berikut:

- Satu Category memiliki banyak Wisata.
- Satu Wisata memiliki banyak Review.
- Satu User dapat memberikan banyak Review.

Tabel User

Berfungsi menyimpan data akun pengguna.

Field	Tipe
id	Integer
username	String
email	String
password	String

Field	Tipe
role	Enum
createdAt	DateTime
updatedAt	DateTime

Tabel Category

Berfungsi menyimpan kategori tempat wisata.

Contoh:

- Pantai
- Gunung
- Air Terjun
- Danau

Field:

Field	Tipe
id	Integer
nama	String

Tabel Wisata

Berfungsi menyimpan informasi objek wisata.

Field:

Field	Tipe
id	Integer
nama	String
deskripsi	String

Field	Tipe
lokasi	String
harga	Integer
gambar	String
categoryId	Integer

Tabel Review

Berfungsi menyimpan ulasan pengguna terhadap wisata.

Field:

Field	Tipe
id	Integer
nama	String
komentar	String
rating	Integer
wisataId	Integer
userId	Integer

3.5 Implementasi Modul Backend

Backend dibangun menggunakan konsep **Module** pada framework NestJS.

Adapun module yang berhasil dibuat yaitu:

- Auth Module
- User Module
- Wisata Module
- Category Module

- Review Module
 - Upload Module
 - Prisma Module
-

3.5.1 Auth Module

Auth Module digunakan untuk proses autentikasi pengguna.

Struktur file:

```
auth
|
├── auth.controller.ts
├── auth.service.ts
├── auth.module.ts
└── dto
    └── login.dto.ts
```

Endpoint:

Method Endpoint

POST /auth/login

3.5.2 Category Module

Category Module digunakan untuk mengelola kategori wisata.

Endpoint yang tersedia:

Method Endpoint

GET /category

POST /category

PATCH /category/:id

DELETE /category/:id

Seluruh endpoint berhasil diimplementasikan menggunakan Prisma ORM.

3.5.3 Wisata Module

Module ini digunakan untuk mengelola data tempat wisata.

Endpoint:

Method Endpoint

GET /wisata

GET /wisata/:id

POST /wisata

PATCH /wisata/:id

DELETE /wisata/:id

Selain itu, data wisata telah dihubungkan dengan tabel Category menggunakan relasi foreign key sehingga setiap wisata dapat memiliki kategori tertentu.

3.5.4 Review Module

Module Review digunakan untuk menyimpan ulasan pengguna terhadap suatu tempat wisata.

Endpoint yang telah dipersiapkan meliputi:

Method Endpoint

GET /review

POST /review

Pengembangan lebih lanjut akan menambahkan fitur edit dan hapus review.

3.5.5 Upload Module

Upload Module dipersiapkan untuk menangani proses upload gambar wisata.

Module ini akan digunakan pada tahap pengembangan berikutnya agar administrator dapat mengunggah foto objek wisata.

3.6 Pengujian API

Pengujian backend dilakukan menggunakan aplikasi **Postman**.

Beberapa endpoint yang berhasil diuji antara lain:

Login

POST /auth/login

Category

GET /category

POST /category

PATCH /category/{id}

DELETE /category/{id}

Wisata

GET /wisata

GET /wisata/{id}

POST /wisata

PATCH /wisata/{id}

DELETE /wisata/{id}

Review

GET /review

POST /review

3.7 Hasil Implementasi

Berdasarkan hasil implementasi, backend aplikasi WisataCui telah berhasil dibangun menggunakan framework NestJS dengan ORM Prisma dan database MySQL. Sistem telah mampu menyediakan layanan REST API untuk pengelolaan data kategori, wisata, pengguna, autentikasi, serta review. Pengujian menggunakan Postman menunjukkan bahwa sebagian besar endpoint telah berjalan sesuai dengan fungsinya sehingga backend siap untuk diintegrasikan dengan frontend pada tahap pengembangan berikutnya.

Database wisataacui di phpMyAdmin

Server: 127.0.0.1 » Database: wisataacui

Filters

Mengandung kata:

Tabel	Tindakan
☐ category	★ Jelajahi Struktur Cari Tambahkan Kosongkan Hapu
☐ review	★ Jelajahi Struktur Cari Tambahkan Kosongkan Hapu
☐ user	★ Jelajahi Struktur Cari Tambahkan Kosongkan Hapu
☐ wisata	★ Jelajahi Struktur Cari Tambahkan Kosongkan Hapu

4 tabel Jumlah

☐ Pilih Semua Dengan pilihan: ▼

Cetak Kamus data

Create new table

Nama tabel

Jumlah kolom

Buat

Konsol

 http://localhost:3000/category  Save

POST

http://localhost:3000/category

Send

ParamsAuthorizationHeaders (8)BodyPre-request ScriptTestsSettingsCookies

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

JSON

Beautify

1

2

3

```
"nama": "Sanggar"
```

BodyCookiesHeaders (8)Test Results

 201 Created 21 ms 376 B [Save Response](#)

PrettyRawPreviewVisualizeJSON

 

1

2

3

4

5

6

```
{
  "id": 4,
  "nama": "Sanggar",
  "createdAt": "2026-06-16T19:24:08.162Z",
  "updatedAt": "2026-06-16T19:24:08.162Z"
}
```

HTTP <http://localhost:3000/category>  Save

GET <http://localhost:3000/category>

Send

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings

Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary JSON

Beautify

```
1  {
2    "nama": "Sanggar"
3  }
```

Body Cookies Headers (8) Test Results

 200 OK 8 ms 373 B [Save Response](#)

Pretty

Raw

Preview

Visualize

JSON



```
1  {
2    "id": 4,
3    "nama": "Sanggar",
4    "createdAt": "2026-06-16T19:24:08.162Z",
5    "updatedAt": "2026-06-16T19:24:08.162Z"
6  }
```

 **http://localhost:3000/category/4**  Save

PATCH 

http://localhost:3000/category/4

Send 

Params Authorization Headers (8) **Body**  Pre-request Script Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary

JSON 

Cookies

Beautify

1

2

3

```
"nama": "Sanggar"
```

Body Cookies Headers (8) Test Results

 200 OK 26 ms 371 B [Save Response](#) 

Pretty Raw Preview Visualize

JSON 



 

1

2

3

4

5

6

```
"id": 4,
"nama": "Sanggar",
"createdAt": "2026-06-16T19:24:08.162Z",
"updatedAt": "2026-06-16T19:25:54.873Z"
```

 **http://localhost:3000/category/4**  Save

DELETE

http://localhost:3000/category/4

Send

ParamsAuthorizationHeaders (8)BodyPre-request ScriptTestsSettingsCookies

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

JSON

Beautify

1

2

3

"nama": "Sanggar"

BodyCookiesHeaders (8)Test Results

 500 Internal Server Error 15 ms 338 B [Save Response](#)

PrettyRawPreviewVisualizeJSON

1

2

3

4

"statusCode": 500,

"message": "Internal server error"